МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №5

з дисципліни **"Функціональне програмування"**

на тему **"ЛІНИВІ ОБЧИСЛЕННЯ: ГЕНЕРАТОРИ ТА ITERTOOLS"**

Варіант 14. Нескінченний генератор шаблонів

Виконала:
Студентка групи ФЕП-23
Поведа Анастасія

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: опанувати генератори (`yield`, `yield from`, генераторні вирази), протокол ітератора та принцип лінивих обчислень; навчитися описувати нескінченні послідовності й безпечно обмежувати їх засобами `itertools`; реалізувати нескінченний генератор шаблонів із порядковою нумерацією елементів і зупинкою за умовою (варіант 14), а також універсальні `take` / `drop` / `sliding_window` з методички.

Теоретичні відомості:

Ітератор — об'єкт, який віддає значення по одному через `__next__` і сигналізує про кінець винятком `StopIteration`. Генераторна функція з `yield` створює такий ітератор автоматично: тіло виконується лише до наступного `yield` і лише тоді, коли споживач запитав значення. Завдяки цьому потік може бути нескінченним — циклічне повторення шаблону чи послідовність станів автомата описуються коротким циклом `while True`, а завершення забезпечує обмежувач на виході: `islice` за кількістю, `takewhile` за умовою або власний `take`. Генератор є одноразовим: після вичерпання його потрібно створити знову.

Модуль `itertools` надає ліниві будівельні блоки: `cycle` нескінченно повторює скінченну послідовність, `count` дає нескінченний лічильник, `zip` поєднує їх у пари «номер — елемент», не обчислюючи нічого наперед, `takewhile` / `dropwhile` відбирають за предикатом, `accumulate` будує префіксні згортки, `starmap` застосовує функцію до кортежів. Ковзне вікно над потоком будується на `deque(maxlen=k)`, яка зберігає лише `k` останніх елементів. Матеріалізацію через `list` або `tuple` виконують лише на фінальному етапі конвеєра.

Хід роботи:

1. У `streams.py` реалізувала базові ліниві потоки з методички: нескінченний `fib_stream`, `naturals` через `yield from count`, утиліти `take` (єдина матеріалізація у список) і `drop` (промотування через `islice`), а також

`sliding_window` на `deque(maxlen=k)`, що не віддає жодного вікна, якщо елементів менше `k`, і піднімає `ValueError` для `k <= 0`:

```python
def fib_stream() -> Iterator[int]:
    """Нескінченний потік чисел Фібоначчі: 0, 1, 1, 2, 3, ..."""
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b


def naturals(start: int = 1) -> Iterator[int]:
    """Нескінченний лічильник натуральних чисел від start (делегує itertools.count)."""
    yield from count(start)


def take(n: int, it: Iterable[T]) -> list[T]:
    """Перші n елементів як список; n < 0 — ValueError, n == 0 — порожній список."""
    if n < 0:
        raise ValueError("n must be >= 0")
    return list(islice(it, n))


def drop(n: int, it: Iterable[T]) -> Iterator[T]:
    """Ітератор без перших n елементів (ліниво «промотує» n значень)."""
    if n < 0:
        raise ValueError("n must be >= 0")
    iterator = iter(it)
    next(islice(iterator, n, n), None)
    return iterator


def sliding_window(it: Iterable[T], k: int) -> Iterator[tuple[T, ...]]:
    """Ковзні вікна розміру k як кортежі; якщо елементів менше k — жодного вікна."""
    if k <= 0:
        raise ValueError("window size must be > 0")
    iterator = iter(it)
    window: deque[T] = deque(islice(iterator, k), maxlen=k)
    if len(window) < k:
        return
    yield tuple(window)
    for x in iterator:
        window.append(x)
        yield tuple(window)

```

2. Додала похідні ліниві операції над вікнами та обгортки `itertools`: `moving_average` як генераторний вираз над вікнами, `window_sums` через `starmap`,

`prefix_sums` / `prefix_products` через `accumulate` і `take_while` / `drop_while`:

```python
def moving_average(it: Iterable[float], k: int) -> Iterator[float]:
    """Рухоме середнє по вікнах розміру k (ліниво, через sliding_window)."""
    return (sum(w) / k for w in sliding_window(it, k))


def window_sums(it: Iterable[float], k: int) -> Iterator[float]:
    """Ковзні суми по вікнах розміру k через starmap(sum, ...)."""
    return starmap(lambda *w: sum(w), sliding_window(it, k))


def prefix_sums(it: Iterable[float]) -> Iterator[float]:
    """Префіксні суми (itertools.accumulate)."""
    return accumulate(it)


def prefix_products(it: Iterable[float]) -> Iterator[float]:
    """Префіксні добутки (accumulate з operator.mul)."""
    return accumulate(it, operator.mul)


def take_while(pred: Callable[[T], bool], it: Iterable[T]) -> Iterator[T]:
    """Елементи, поки предикат істинний (itertools.takewhile)."""
    return takewhile(pred, it)


def drop_while(pred: Callable[[T], bool], it: Iterable[T]) -> Iterator[T]:
    """Елементи після першого, для якого предикат хибний (itertools.dropwhile)."""
    return dropwhile(pred, it)
```

3. За варіантом 14 у `patterns.py` описала елемент потоку `Numbered` (`NamedTuple` з порядковим номером, символом і номером циклу), власний нескінченний генератор `repeat_pattern` (`yield from` шаблону у циклі `while True`; порожній шаблон відхиляється) та `numbered`, який поєднує `count(start)` і `cycle(pattern)` через `zip` у генераторному виразі:

```python
class Numbered(NamedTuple, Generic[T]):
    """Елемент шаблону з порядковим номером і номером циклу повторення."""

    seq: int
    item: T
    lap: int


def repeat_pattern(pattern: Iterable[T]) -> Iterator[T]:
    """Нескінченно повторює шаблон (власний генератор з yield from).

    Порожній шаблон — ValueError, інакше цикл ніколи б не віддав значення.
    """
    items = tuple(pattern)
    if not items:
        raise ValueError("pattern must not be empty")
    while True:
        yield from items


def numbered(pattern: Iterable[T], start: int = 1) -> Iterator[Numbered[T]]:
    """Нескінченний потік (номер, символ, цикл): zip(count(start), cycle(pattern))."""
    items = tuple(pattern)
    if not items:
        raise ValueError("pattern must not be empty")
    return (
        Numbered(seq, item, (seq - start) // len(items))
        for seq, item in zip(count(start), cycle(items), strict=False)
    )
```

4. Реалізувала способи зупинки нескінченного потоку: `until` — за довільною умовою через `takewhile` з інвертованим предикатом, `first_n` — за кількістю через `islice` (єдине місце матеріалізації), `laps` — за кількістю повних повторень

шаблону; додала генератор станів автомата `states` за таблицею переходів і `pattern_windows` — ковзні вікна над циклічним шаблоном через `sliding_window`:

```python
def until(
    stop: Callable[[Numbered[T]], bool], it: Iterable[Numbered[T]]
) -> Iterator[Numbered[T]]:
    """Елементи, поки умова зупинки хибна (takewhile з інвертованим предикатом)."""
    return takewhile(lambda x: not stop(x), it)


def first_n(pattern: Iterable[T], n: int, start: int = 1) -> list[Numbered[T]]:
    """Перші n пронумерованих елементів (єдина матеріалізація — на виході)."""
    if n < 0:
        raise ValueError("n must be >= 0")
    return list(islice(numbered(pattern, start), n))


def laps(pattern: Iterable[T], max_laps: int) -> Iterator[Numbered[T]]:
    """Рівно max_laps повних повторень шаблону (зупинка за номером циклу)."""
    if max_laps < 0:
        raise ValueError("max_laps must be >= 0")
    return until(lambda x: x.lap >= max_laps, numbered(pattern))


def states(transitions: Mapping[T, T], start: T) -> Iterator[T]:
    """Нескінченний потік станів автомата: start, transitions[start], ...

    Власний генератор з yield; наступний стан обчислюється лише за запитом.
    """
    state = start
    while True:
        yield state
        state = transitions[state]


def pattern_windows(pattern: Iterable[T], k: int) -> Iterator[tuple[T, ...]]:
    """Ковзні вікна розміру k над нескінченним повторенням шаблону."""
    return sliding_window(repeat_pattern(pattern), k)
```

5. У `main.py` зробила демонстраційний запуск: перші значення Фібоначчі, `drop` + `take`, віконні суми натуральних чисел, рухоме середнє, префіксні суми, а далі пронумерований шаблон `ABC`, два повні цикли, зупинка за умовою, вікна, світлофор як автомат станів і пропуск ста елементів нескінченного потоку:

```python
PATTERN = ("A", "B", "C")
LIGHTS = {"red": "green", "green": "yellow", "yellow": "red"}

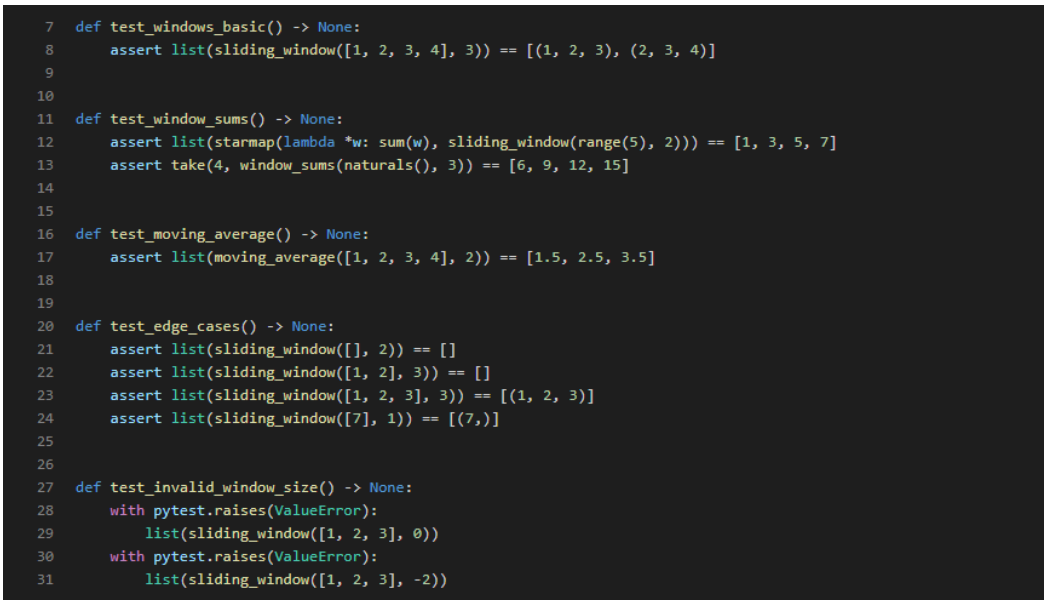

def main() -> None:
    """Друкує приклади роботи потоків (єдиний I/O у лабораторній)."""
    print("fib first 10:", take(10, fib_stream()))
    print("fib drop 5 take 8:", take(8, drop(5, fib_stream())))
    print(
        "window sums of naturals (k=3, first 10):", take(10, window_sums(naturals(), 3))
    )
    print("moving average [1..6], k=2:", list(moving_average([1, 2, 3, 4, 5, 6], 2)))
    print("prefix sums [1,2,3,4]:", list(prefix_sums([1, 2, 3, 4])))

    print("first 7 of ABC:", [f"{x.seq}:{x.item}" for x in first_n(PATTERN, 7)])
    print("2 laps of ABC :", [(x.seq, x.item, x.lap) for x in laps(PATTERN, 2)])
    stopped = until(lambda x: x.seq > 4 and x.item == "A", numbered(PATTERN))
    print("until A after 4:", [x.item for x in stopped])
    print("windows k=2  :", take(4, pattern_windows(PATTERN, 2)))
    print("lights       :", take(7, states(LIGHTS, "red")))
    print("drop 100, 3  :", [x.seq for x in islice(drop(100, numbered(PATTERN)), 3)])


if __name__ == "__main__":
    main()
```

6. Написала тести для вікон у `tests/test_sliding_window.py`: базові вікна, суми через `starmap`, рухоме середнє та граничні випадки — порожнє джерело, `k`

більше за довжину, вікно з одного елемента, некоректний розмір вікна:



```python
def test_windows_basic() -> None:
    assert list(sliding_window([1, 2, 3, 4], 3)) == [(1, 2, 3), (2, 3, 4)]


def test_window_sums() -> None:
    assert list(starmap(lambda *w: sum(w), sliding_window(range(5), 2))) == [1, 3, 5, 7]
    assert take(4, window_sums(naturals(), 3)) == [6, 9, 12, 15]


def test_moving_average() -> None:
    assert list(moving_average([1, 2, 3, 4], 2)) == [1.5, 2.5, 3.5]


def test_edge_cases() -> None:
    assert list(sliding_window([], 2)) == []
    assert list(sliding_window([1, 2], 3)) == []
    assert list(sliding_window([1, 2, 3], 3)) == [(1, 2, 3)]
    assert list(sliding_window([7], 1)) == [(7,)]


def test_invalid_window_size() -> None:
    with pytest.raises(ValueError):
        list(sliding_window([1, 2, 3], 0))
    with pytest.raises(ValueError):
        list(sliding_window([1, 2, 3], -2))
```

7. У `tests/test_patterns.py` перевірила циклічне повторення, нумерацію і номери циклів (зокрема зі зміщеним стартом), зупинку за умовою, точну

кількість повторень у `laps`, автомат станів і вікна над шаблоном, у тому числі ширші за сам шаблон:

```python
def test_repeat_pattern_cycles(pattern: list[str]) -> None:
    assert take(7, repeat_pattern(pattern)) == ["A", "B", "C", "A", "B", "C", "A"]
    assert take(3, repeat_pattern("x")) == ["x", "x", "x"]


def test_numbered_sequence_and_laps(pattern: list[str]) -> None:
    assert first_n(pattern, 4) == [
        Numbered(1, "A", 0),
        Numbered(2, "B", 0),
        Numbered(3, "C", 0),
        Numbered(4, "A", 1),
    ]
    assert [x.seq for x in first_n(pattern, 3, start=10)] == [10, 11, 12]
    assert [x.lap for x in first_n(pattern, 7)] == [0, 0, 0, 1, 1, 1, 2]


def test_until_stops_by_condition(pattern: list[str]) -> None:
    stopped = until(lambda x: x.seq > 4 and x.item == "A", numbered(pattern))
    assert [x.item for x in stopped] == ["A", "B", "C", "A", "B", "C"]
    assert list(until(lambda x: True, numbered(pattern))) == []


def test_laps_exact_repetitions(pattern: list[str]) -> None:
    assert [x.item for x in laps(pattern, 2)] == ["A", "B", "C", "A", "B", "C"]
    assert list(laps(pattern, 0)) == []
    assert len(list(laps("ab", 5))) == 10


def test_states_machine() -> None:
    assert take(5, states(LIGHTS, "red")) == ["red", "green", "yellow", "red", "green"]
    assert take(3, states({1: 1}, 1)) == [1, 1, 1]


def test_pattern_windows(pattern: list[str]) -> None:
    assert take(4, pattern_windows(pattern, 2)) == [
        ("A", "B"),
        ("B", "C"),
        ("C", "A"),
        ("A", "B"),
    ]
    assert take(2, pattern_windows(pattern, 5)) == [
        ("A", "B", "C", "A", "B"),
        ("B", "C", "A", "B", "C"),
    ]

```

8. Додала крайні випадки (порожній шаблон, `n = 0`, `n < 0`, від'ємна кількість циклів, `k = 0`) і тест, що зміна вхідного списку після створення потоку не впливає на нього, бо шаблон копіюється у `tuple`:

```python
def test_edge_cases(pattern: list[str]) -> None:
    assert first_n(pattern, 0) == []
    assert take(3, repeat_pattern(["only"])) == ["only"] * 3
    with pytest.raises(ValueError):
        list(repeat_pattern([]))
    with pytest.raises(ValueError):
        numbered([])
    with pytest.raises(ValueError):
        first_n(pattern, -1)
    with pytest.raises(ValueError):
        list(laps(pattern, -1))
    with pytest.raises(ValueError):
        list(pattern_windows(pattern, 0))


def test_pattern_not_mutated_and_independent() -> None:
    pattern = ["A", "B"]
    stream = numbered(pattern)
    assert next(stream).item == "A"
    pattern.append("Z")  # зміна списку після старту не впливає на потік
    assert [x.item for x in islice(stream, 3)] == ["B", "A", "B"]
    assert pattern == ["A", "B", "Z"]

```

9. Три property-тести на Hypothesis: для випадкових шаблонів елемент із номером `seq` завжди дорівнює `pattern[(seq - 1) % len]`, `laps(pattern, k)` дає рівно `pattern * k`, а вікна над шаблоном є послідовними зрізами розгорнутого потоку:

```python
patterns = st.lists(st.text(min_size=1, max_size=3), min_size=1, max_size=6)


@given(patterns, st.integers(min_value=0, max_value=50))
def test_property_item_is_pattern_at_seq_mod_len(pat: list[str], n: int) -> None:
    for x in first_n(pat, n):
        assert x.item == pat[(x.seq - 1) % len(pat)]
        assert x.lap == (x.seq - 1) // len(pat)


@given(patterns, st.integers(min_value=0, max_value=10))
def test_property_laps_length(pat: list[str], k: int) -> None:
    items = [x.item for x in laps(pat, k)]
    assert items == pat * k


@given(patterns, st.integers(min_value=1, max_value=5))
def test_property_windows_are_consecutive(pat: list[str], k: int) -> None:
    source = take(k + 10, repeat_pattern(pat))
    windows = take(11, pattern_windows(pat, k))
    assert windows == [tuple(source[i : i + k]) for i in range(11)]
```

10. У `tests/test_laziness.py` підтвердила лінивість: ітератор-лічильник `CountingSource` показує, що `take`, `sliding_window` і конвеєр `moving_average(drop(…))` беруть рівно стільки елементів, скільки спожито; таблиця переходів із лічильником доводить, що `states` обчислює наступний

стан лише за запитом, а потік із мільйоном пропущених елементів не матеріалізується:

```python
class CountingSource:
    """Ітератор, що рахує, скільки елементів у нього реально запитали."""

    def __init__(self) -> None:
        self.pulled = 0

    def __iter__(self) -> Iterator[int]:
        return self

    def __next__(self) -> int:
        self.pulled += 1
        return self.pulled


def test_take_pulls_exactly_n() -> None:
    src = CountingSource()
    assert take(4, src) == [1, 2, 3, 4]
    assert src.pulled == 4


def test_sliding_window_pulls_only_what_is_consumed() -> None:
    src = CountingSource()
    windows = sliding_window(src, 3)
    assert src.pulled == 0  # генератор ще не стартував
    assert next(windows) == (1, 2, 3)
    assert src.pulled == 3
    assert next(windows) == (2, 3, 4)
    assert src.pulled == 4


def test_pipeline_is_lazy_end_to_end() -> None:
    src = CountingSource()
    pipeline = moving_average(drop(2, src), 2)
    assert src.pulled == 2  # drop промотує n елементів одразу, вікна ще не стартували
    assert list(islice(pipeline, 2)) == [3.5, 4.5]
    assert src.pulled == 5  # 2 пропущено + 3 для двох вікон по 2


def test_states_computed_on_demand_only() -> None:
    calls = {"n": 0}

    class Transitions(dict[int, int]):
        def __getitem__(self, key: int) -> int:
            calls["n"] += 1
            return key + 1

    stream = states(Transitions(), 0)
    assert calls["n"] == 0
    assert list(islice(stream, 4)) == [0, 1, 2, 3]
    assert calls["n"] == 3  # переходів рівно на один менше за спожиті стани


def test_infinite_pattern_terminates_only_via_limiter() -> None:
    stream = until(lambda x: x.seq > 1000, numbered("ab"))
    assert len(list(stream)) == 1000
    assert take(1, drop(10**6, repeat_pattern("ab"))) == ["a"]
```

11. Запустила демонстрацію: усі нескінченні джерела (Фібоначчі, натуральні числа, циклічний шаблон, автомат станів) обмежуються `take` / `islice` /

`takewhile`, тому обробляються без зависання і без зайвої пам'яті:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab05> python main.py
fib first 10: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
fib drop 5 take 8: [5, 8, 13, 21, 34, 55, 89, 144]
window sums of naturals (k=3, first 10): [6, 9, 12, 15, 18, 21, 24, 27, 30, 33]
moving average [1..6], k=2: [1.5, 2.5, 3.5, 4.5, 5.5]
prefix sums [1,2,3,4]: [1, 3, 6, 10]
first 7 of ABC: ['1:A', '2:B', '3:C', '4:A', '5:B', '6:C', '7:A']
2 laps of ABC : [(1, 'A', 0), (2, 'B', 0), (3, 'C', 0), (4, 'A', 1), (5, 'B', 1), (6, 'C', 1)]
until A after 4: ['A', 'B', 'C', 'A', 'B', 'C']
windows k=2  : [('A', 'B'), ('B', 'C'), ('C', 'A'), ('A', 'B')]
lights       : ['red', 'green', 'yellow', 'red', 'green', 'yellow', 'red']
drop 100, 3  : [101, 102, 103]
```

12. Перевірила проєкт автоматичними інструментами: усі 31 тест pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають зауважень до стилю:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab05> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs_functional_for_nastia
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 31 items

tests\test_accumulate.py::test_accumulate_sum_prod PASSED                [  3%]
tests\test_accumulate.py::test_accumulate_max_and_empty PASSED           [  6%]
tests\test_accumulate.py::test_takewhile_dropwhile_on_infinite PASSED    [  9%]
tests\test_fib_stream.py::test_first_10 PASSED                           [ 12%]
tests\test_fib_stream.py::test_drop_then_take PASSED                     [ 16%]
tests\test_fib_stream.py::test_stream_is_fresh_each_call PASSED          [ 19%]
tests\test_laziness.py::test_take_pulls_exactly_n PASSED                 [ 22%]
tests\test_laziness.py::test_sliding_window_pulls_only_what_is_consumed PASSED [ 25%]
tests\test_laziness.py::test_pipeline_is_lazy_end_to_end PASSED          [ 29%]
tests\test_laziness.py::test_states_computed_on_demand_only PASSED       [ 32%]
tests\test_laziness.py::test_infinite_pattern_terminates_only_via_limiter PASSED [ 35%]
tests\test_patterns.py::test_repeat_pattern_cycles PASSED                [ 38%]
tests\test_patterns.py::test_numbered_sequence_and_laps PASSED           [ 41%]
tests\test_patterns.py::test_until_stops_by_condition PASSED             [ 45%]
tests\test_patterns.py::test_laps_exact_repetitions PASSED               [ 48%]
tests\test_patterns.py::test_states_machine PASSED                       [ 51%]
tests\test_patterns.py::test_pattern_windows PASSED                      [ 54%]
tests\test_patterns.py::test_edge_cases PASSED                           [ 58%]
tests\test_patterns.py::test_pattern_not_mutated_and_independent PASSED  [ 61%]
tests\test_patterns.py::test_property_item_is_pattern_at_seq_mod_len PASSED [ 64%]
tests\test_patterns.py::test_property_laps_length PASSED                 [ 67%]
tests\test_patterns.py::test_property_windows_are_consecutive PASSED     [ 70%]
tests\test_sliding_window.py::test_windows_basic PASSED                  [ 74%]
tests\test_sliding_window.py::test_window_sums PASSED                    [ 77%]
tests\test_sliding_window.py::test_moving_average PASSED                 [ 80%]
tests\test_sliding_window.py::test_edge_cases PASSED                     [ 83%]
tests\test_sliding_window.py::test_invalid_window_size PASSED            [ 87%]
tests\test_take_drop.py::test_take_advances_shared_iterator PASSED       [ 90%]
tests\test_take_drop.py::test_drop_then_take_on_count PASSED             [ 93%]
tests\test_take_drop.py::test_edge_cases PASSED                          [ 96%]
tests\test_take_drop.py::test_negative_n_rejected PASSED                 [100%]

============================== 31 passed in 0.62s ==============================

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab05> python -m mypy .
Success: no issues found in 10 source files

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab05> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab05> python -m black --check .
All done! ✨ 🍰 ✨
10 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я реалізувала ліниві потоки на генераторах та `itertools`: нескінченні `fib_stream`, `naturals`, універсальні `take` / `drop` / `sliding_window` на `deque`, а за варіантом 14 — нескінченний генератор шаблонів, який циклічно повторює набір символів або станів (`cycle`, власний `yield from`), нумерує елементи через `zip` із `count` і зупиняється за умовою (`takewhile`), за кількістю (`islice`) або за числом повторень. Жодна функція ядра не матеріалізує потік, що підтверджено тестами лінивості з лічильниками звернень; граничні випадки, property-тести на Hypothesis та перевірки mypy, ruff і black підтвердили коректність реалізації.